IMPORTANT DATES

Personal reminder application — product requirements & technical blueprint

Purpose	A private, installable app for remembering birthdays, anniversaries, memorial dates and personally meaningful milestones.
Users	The owner plus only people explicitly invited by their email address.
Devices	Android phone and laptop, with one account and synchronized access.
Cost goal	No paid domain and no paid hosting at the initial scale; use free tiers where practical.

Product principle: The app should make important dates easy to enter, safe to share with invited people, and dependable to receive reminders for—even when the app is not open.

Prepared as a build specification. This document describes the intended product and a recommended implementation; it is not a deployed application.

1. The idea

Build a small personal “important dates” application that replaces a handwritten list. The owner can enter dates once, organize them, and receive reminders every year or on a chosen one-time date. The application should work as an installable app on an Android phone and laptop.

The application is not limited to birthdays. It should support any date the user chooses to remember, including anniversaries, memorial/death anniversaries, first meetings, achievements, personal milestones and other private events.

2. Goals and non-goals

Goals

• Add, edit, archive and delete important dates.

• Support annual recurrence and one-time events.

• Allow the owner to invite specific people by email; no public sign-up.

• Synchronize authorized users across Android and laptop.

• Send configurable reminders before the event and on the event date.

• Keep the interface simple, responsive, accessible and easy to back up.

Non-goals for the first version

• No social network, public profiles or public event discovery.

• No paid domain, app-store publication or native Android APK requirement.

• No complicated contact-book import or messaging integrations in the first release.

3. Main user experience

Dashboard

• Show today’s events, upcoming events, and the next 30 days.

• Show event category, person/name, date and next reminder.

• Provide quick actions: Add date, Search, Calendar, Manage access and Settings.

Add / edit an event

• Title or person’s name.

• Event type: Birthday, Anniversary, Memorial, Relationship, Milestone, or Custom.

• Date (day, month and year where known).

• Repeat: Never or Every year.

• Optional notes; optional relationship or label.

• Visibility: Shared with invited members or private to the owner (recommended).

• Reminder settings: e.g. 1 day before and on the date; allow custom choices.

Calendar and search

• Month view and chronological list view.

• Filter by category and search by name/title.

- Clearly distinguish recurring dates from one-time dates.

4. Access control: invited people only

The owner controls who can use the application. Access is granted to specific email addresses, not to anyone who finds the URL.

Recommended invitation flow

1. Owner	Enters a person’s email address in “Manage access” and sends an invitation.
2. Invitee	Opens the app and signs in using the invited email (Google sign-in or email one-time link/code).
3. Verification	The backend checks that the authenticated email has an active invitation/allowlist entry.
4. Access	If authorized, the person enters the app. If not authorized, access is denied.
5. Owner control	The owner can revoke access. The next protected request is denied.

Important: Hiding the sign-up button is not security. Every database operation must enforce authorization on the server. Use database row-level security (RLS) and an allowlist/invitation table. Never rely only on frontend checks.

Suggested roles and permissions

Role	Permissions
Owner	Invite/revoke members; manage all shared events; manage settings; export/backup data.
Invited member	View and receive reminders for shared events; optionally add/edit shared events if the owner enables this.

Recommended privacy model

Use one shared household/workspace for events intended for all invited members, while allowing owner-only private events. This avoids accidentally exposing sensitive personal notes or dates to every invited person. Decide later whether members may create events or only view them.

5. How reminders should work

• An event can have one or more reminder offsets, such as 7 days before, 1 day before, and on the date.

• Annual events recur automatically without re-entering them.

• The system calculates the next occurrence using the event’s local date and timezone.

• A scheduled server-side job checks due reminders and sends push notifications.

• Store a delivery record/idempotency key so the same reminder is not sent repeatedly.

• Each authorized user can subscribe their phone/laptop browser to push notifications.

Reliability caveat: Push notifications require the user to grant permission and the device/browser to support them. Android Chrome generally supports installed-PWA web push. Laptop support depends on the browser and operating system. Battery saver, notification settings, connectivity and browser restrictions can delay or suppress alerts. The app should also show an in-app “upcoming reminders” list as a fallback.

6. Recommended technical architecture

Keep the first version as a Progressive Web App (PWA). It can be installed from the browser on Android and supported desktop browsers, without publishing to an app store.

Layer	Recommended choice	Purpose
Frontend	React + Vite + TypeScript	Responsive interface, forms, calendar and dashboard.
Installability	PWA manifest + service worker	Home-screen install, app icon, caching and push event handling.
Hosting	GitHub Pages	Host the static frontend at a free GitHub Pages URL.
Authentication	Supabase Auth	Google sign-in or email OTP/magic link.
Database	Supabase PostgreSQL	Store users, invitations, events, preferences and delivery logs.
Authorization	RLS + server-side checks	Restrict data to owner and approved members.
Reminder scheduler	Scheduled function / cron	Find reminders that are due and queue deliveries.
Push	Web Push + service worker	Deliver browser push notifications to subscribed devices.

System flow

Android PWA and **Laptop browser/PWA**
HTTPS connection • sign-in • event management • push subscription
GitHub Pages Static React application
Supabase Auth → invitation/allowlist check → PostgreSQL with RLS
Scheduled reminder worker → due reminder check → Web Push service
Service workers on each device → display notification

7. Data model (initial version)

Table	Important fields	Why it exists
profiles	user_id, display_name, email	User identity and profile.
memberships / invitations	email, role, status, invited_by, accepted_at	Allow only invited email addresses; support revocation.
events	id, title, category, event_date, recurrence, visibility, notes, created_by	Store birthdays and all other important dates.
event_access	event_id, user_id, permission	Control which invited members can see/edit an event, if per-event sharing is used.
push_subscriptions	user_id, endpoint, keys, device_label	Remember each browser/device that can receive push.

Table	Important fields	Why it exists
reminder_deliveries	event_id, user_id, occurrence_date, reminder_offset, sent_at, status	Prevent duplicate alerts and record delivery status.

8. Hosting, installation and cost

No custom domain is required. GitHub Pages can host the frontend on a github.io address. Users can install the PWA from that HTTPS site. The backend still needs a hosted service for authentication, database access and scheduled push delivery.

Cost expectations

• GitHub repository and GitHub Pages: free for eligible public repositories; GitHub Pages plan and usage rules should be checked before deployment.

• Supabase: has a free tier, but quotas, pausing rules and feature availability can change. Check current limits before relying on it.

• Domain: optional; use the GitHub Pages URL instead.

• Push notifications: browser web push itself does not require buying a domain, but requires HTTPS, a push subscription and a server-side sender. Hosting/scheduler usage must remain within free-tier limits.

Important distinction: GitHub Pages alone cannot run a reliable background scheduler or securely send push notifications. The frontend can be free on GitHub Pages, while a backend service is needed for login, database security and reminders.

Installation

• Android: open the deployed HTTPS URL in Chrome, choose “Install app” or “Add to Home screen”, then allow notifications.

• Laptop: open the URL in a supported browser (such as Chrome or Edge) and use its Install option where available.

• Sign in with the same authorized email on both devices. Each device/browser must separately grant notification permission and register its push subscription.

9. Security and privacy requirements

• Do not put secret keys, service-role keys or private credentials in frontend code or GitHub.

• Use HTTPS, secure authentication and database row-level security.

• Validate invitation status on the backend for every protected action.

• Allow the owner to revoke a member; revocation must take effect on subsequent requests.

• Do not expose private notes or owner-only events to invited members.

• Provide export and deletion options. Back up important data periodically.

• Collect only the information needed. Treat dates and notes as personal information.

10. Build plan

Phase	Deliverable	Acceptance check
1. Foundation	React app, responsive layout, GitHub repository and Pages deployment.	App opens on phone and laptop over HTTPS.
2. Events	Add/edit/delete dates, categories, recurrence and calendar/list views.	Annual dates calculate the next occurrence correctly.
3. Accounts	Supabase Auth, owner account, invitation allowlist and member roles.	Uninvited email is denied; invited email can sign in.
4. Shared data	Database, RLS, shared/private visibility and access revocation.	Members see only permitted events and cannot bypass access rules.

Phase	Deliverable	Acceptance check
5. Reminders	Scheduler, Web Push, service worker and device subscriptions.	Test reminders arrive on Android and laptop with app closed, under supported conditions.
6. Hardening	Export/restore, duplicate prevention, error handling and documentation.	Data can be backed up and restored; duplicate notifications are prevented.

11. Decisions to confirm before implementation

The current requirements establish the product direction. These details should be finalized before database and notification code is written:

• **Shared calendar:** Should invited members see all shared events, or should the owner choose recipients separately for each event? Recommended: shared calendar plus owner-only private events.

• **Member permissions:** Should invited members only view dates, or may they add/edit shared events? Recommended for v1: owner manages dates; members view, with an optional permission to contribute later.

• **Reminder defaults:** Recommended: 1 day before and on the date, configurable per event.

• **Login method:** Recommended: Google sign-in for convenience, with an email-link option if needed.

• **Leap-day birthdays:** Decide whether a 29 February annual event is observed on 28 February or 1 March in non-leap years.

12. Definition of done

• Owner can sign in and invite specific email addresses.

• Only active invited accounts can access protected application data.

• Owner can revoke an invitation/member.

• Owner can create birthdays, anniversaries, memorial dates and custom dates.

• Annual recurrence and one-time dates work correctly across year boundaries.

• Private events and notes are not visible to other members.

• The PWA can be installed on Android and laptop.

• Push reminders can be tested on both devices, with permission granted and browser/device support enabled.

• Data can be exported and restored.

Recommended first release

Build a private, invite-only PWA with an owner-managed shared calendar, owner-only private events, annual recurrence, two default reminder times, and export. Start with a small number of test events and test push notifications on both devices before entering the complete paper list.

Final note

This is a practical product specification, not a promise that every service will remain free forever. Free plans have quotas and can change. The design avoids a paid domain and paid app-store distribution, while acknowledging that dependable push reminders require a backend and browser/device permissions.